

Theorem Discovery Dynamics: Research Plan

Quantifying the Adjacent Possible in Formal Mathematics

Author: [Your name]

Date: 2026-02-07

Version: 1.0

I. Core Research Question

Not: "How many citations will theorem T accumulate?"

But: "What structural role does T play in expanding the space of discoverable mathematics?"

The Fundamental Tension

Citation count conflates:

- Mathematical necessity (structural role in proof space)
- Social dynamics (research fashion, accessibility)
- Temporal accidents (when related problems become fashionable)

At radius $r=0$, we cannot predict social factors. But we *can* predict structural impact through information-theoretic measures.

II. Current Approach Limitations

What We're Measuring Now

- $\boxed{Y1}$: Total downstream descendants (citation proxy)
- Features: Local graph topology (depth, parent stats, parentset structure)

Performance at $r=0$

- $R^2 \approx 0.005$ (essentially zero signal)
- Classification PR-AUC ≈ 0.64 (modest improvement over random)

Why This Fails

1. **Target Mismatch:** Descendants measure *popularity*, not *compression value* or *frontier expansion*
2. **Feature Incompleteness:** Missing proof-level information (tactic usage, AST structure)

3. **No Temporal Dynamics:** Treats DAG as static; ignores growth patterns
 4. **Ignores Conceptual Distance:** Module labels too coarse; need semantic proof similarity
-

III. Minimum Description Length Framework

A. Theoretical Foundation

Core Principle: A theorem's value = how much it compresses the description of all downstream mathematics.

Given:

- Theorem library $L = \{T_1, T_2, \dots, T_n\}$
- Proof corpus $P = \{\text{proof}(T_i) : T_i \in L\}$
- New theorem T^*

Compression Value:

$$\Delta_MDL(T^*) = DL(P \mid L) - DL(P \mid L \cup \{T^*\})$$

Where $DL(P \mid L)$ = description length of proof corpus given library L .

B. Practical Computation

Step 1: Proof Encoding Scheme

Each proof is a sequence of tactics applied to proof states:

$$\text{proof}(T) = [(\text{tactic}_1, \text{premises}_1), (\text{tactic}_2, \text{premises}_2), \dots, (\text{tactic}_k, \text{premises}_k)]$$

Encoding Components:

- Tactic Vocabulary:** $V_tactic = \{\text{rewrite, simp, apply, induction, ...}\}$
 - Size: $|V_tactic| \approx 200$ core tactics in Lean
 - Encoding: $\log_2(|V_tactic|)$ bits per tactic
- Premise Selection:** For tactic i , $\text{premises}_i \subseteq L$ (available lemmas)
 - Encoding: $\log_2(|L_available|)$ bits per premise reference
 - Average premises per step: $\mu_premises \approx 2-3$
- Proof Length:** Number of tactic applications
 - Variable compression: use arithmetic coding based on empirical length distribution

Total Description Length (without compression):

$$DL_raw(\text{proof}(T)) = \sum_i [\log_2(|V_tactic|) + n_i \cdot \log_2(|L_available|)]$$

where n_i = number of premises in tactic i .

Step 2: Incorporating Theorem T^* as Compressor

Algorithmic Procedure:

For each theorem T_j proved *after* T^* in the DAG:

1. **Original Proof:** $\text{proof_original}(T_j)$ using library L
 - Length: $\ell_original$ = number of tactic steps
 - DL: $DL_original$ = description length as above
2. **Alternative Proof:** $\text{proof_alt}(T_j)$ using $L \cup \{T^*\}$
 - This is *hypothetical* - we need to search for it
 - **Approximation Method A** (Conservative):
 - Identify where $\text{proof_original}(T_j)$ produces subgoals solvable by T^*
 - Replace those subproofs with single apply T^* step
 - Count reduction in tactic steps
 - **Approximation Method B** (LLM-assisted):
 - Use proof synthesis model (e.g., GPT-4 + Lean, DeepSeek-Prover)
 - Prompt: "Prove T_j given library $L \cup \{T^*\}$ "
 - Compare generated proof length to original
 - **Approximation Method C** (Subgraph Isomorphism):
 - Build tactic dependency graph for $\text{proof}(T^*)$
 - Search for isomorphic subgraphs in downstream proofs
 - Each match = potential compression site

3. Compression Achieved:

$$\text{compress}(T^*, T_j) = DL_original(T_j) - DL_alt(T_j)$$

4. Aggregate Compression Value:

$$\Delta_MDL(T^*) = \sum_j \text{compress}(T^*, T_j) - DL(T^*)$$

(Subtract theorem's own description length - can't compress more than it costs)

Step 3: Practical Implementation (Given Your Data)

What You Have:

- Complete proofs: tactic sequences + premise references
- Dependency DAG: which theorems use which as lemmas

Computational Strategy:

python

```

def compute_mdl_compression(theorem_T, dag, all_proofs):
    """
    Compute compression value of theorem T.

    Parameters:
    - theorem_T: Target theorem
    - dag: Dependency graph (who uses whom)
    - all_proofs: Dict[theorem_id -> proof_structure]

    Returns:
    - compression_value: Total bits saved by T's existence
    """

    # 1. Get all downstream theorems (descendants in DAG)
    descendants = dag.get_descendants(theorem_T)

    # 2. For each descendant, compute compression
    total_compression = 0

    for desc in descendants:
        # Get proof of descendant
        proof = all_proofs[desc]

        # Method: Subgraph isomorphism in tactic trees
        # Build tactic dependency tree for theorem_T
        T_tactic_tree = build_tactic_tree(all_proofs[theorem_T])

        # Build tactic dependency tree for descendant
        desc_tactic_tree = build_tactic_tree(proof)

        # Find isomorphic subgraphs (potential compression sites)
        matches = find_subgraph_isomorphisms(
            pattern=T_tactic_tree,
            target=desc_tactic_tree,
            match_tactics=True, # Tactics must match
            match_topology=True # Dependency structure must match
        )

        # Each match represents a sequence of tactics replaceable by single "apply T"
        for match in matches:
            # Original cost: all tactics in matched subgraph
            original_cost = sum(
                encoding_cost(tactic, premises, available_library)

```

```

        for tactic, premises in match.nodes
    )

    # New cost: single "apply theorem_T" step
    new_cost = encoding_cost("apply", [theorem_T], available_library)

    # Compression achieved
    total_compression += (original_cost - new_cost)

    # 3. Subtract theorem's own description length
    theorem_cost = compute_proof_length(all_proofs[theorem_T])

    net_compression = total_compression - theorem_cost

    return {
        'gross_compression': total_compression,
        'theorem_cost': theorem_cost,
        'net_compression': net_compression,
        'num_compression_sites': len(matches),
        'descendants_compressed': len([d for d in descendants if matches_found(d)])
    }

```

Tactic Tree Structure:

```

python

class TacticNode:
    def __init__(self, tactic_name, premises, subgoals_generated):
        self.tactic = tactic_name
        self.premises = premises # Which theorems/lemmas used
        self.children = [] # Tactics applied to resulting subgoals

    def signature(self):
        """Canonical representation for isomorphism matching."""
        return (
            self.tactic,
            frozenset(p.theorem_type for p in self.premises), # Types, not IDs
            tuple(sorted(c.signature() for c in self.children))
        )

```

Step 4: Encoding Cost Function

```

python

```

```
def encoding_cost(tactic_name, premises, available_library):
    """
    Compute description length in bits for one tactic application.

    Uses Shannon entropy of empirical tactic distribution.
    """
    # Tactic selection cost
    tactic_prob = empirical_tactic_distribution[tactic_name]
    tactic_cost = -log2(tactic_prob) # Shannon optimal code

    # Premise selection cost
    # For n premises from library of size L: log2(C(L, n))
    L = len(available_library)
    n = len(premises)
    premise_cost = log2(comb(L, n)) if n > 0 else 0

    # Additional: premise ordering (if order matters)
    ordering_cost = log2(factorial(n)) if n > 1 else 0

    return tactic_cost + premise_cost + ordering_cost
```

Refinement: Use empirical distributions from corpus:

- $P(\text{tactic} \mid \text{proof_context})$: Tactic probability given proof state features
- $P(\text{premise} \mid \text{tactic}, \text{goal})$: Premise selection probability
- Conditional encoding based on n-gram models of proof sequences

C. Tractable Approximation for Large Scale

Full subgraph isomorphism is NP-hard. **Fast approximation:**

1. Hash-based Matching:

python

```

# Create hash signatures for tactic subtrees
def hash_subtree(node, depth=3):
    if depth == 0:
        return hash(node.tactic)
    return hash((
        node.tactic,
        tuple(sorted(hash_subtree(c, depth-1) for c in node.children))
    ))

# Build index: hash -> list of (theorem_id, subtree_location)
hash_index = defaultdict(list)
for theorem in library:
    for subtree in extract_subtrees(theorem.proof):
        h = hash_subtree(subtree)
        hash_index[h].append((theorem.id, subtree))

# For new theorem T, check if its subtrees appear downstream
compression_sites = []
for subtree in extract_subtrees(T.proof):
    h = hash_subtree(subtree)
    compression_sites.extend(hash_index[h])

```

2. Proof Sketch Embedding:

- Embed each proof as vector: proof2vec (train on tactic sequences)
- Cosine similarity > threshold → candidate for compression
- Verify with lightweight alignment algorithm

IV. Alternative Prediction Targets

Move beyond raw descendant count. Predict structural impact measures.

Target 1: MDL Compression Value

Definition: $\Delta_MDL(T)$ as computed above

Why Better:

- Captures *utility* not popularity
- Theorems that enable shorter proofs have high value even with few direct citations
- Aligns with "elegance" intuition (Occam's razor)

Prediction at r=0: Extract features from T's proof structure:

- Generality: Type signature abstractness (uses type classes vs. concrete types)
- Reusability: Tactic pattern frequency (common patterns → more reuse)
- Bridging: Combines premises from distant modules → connects subfields

Target 2: Betweenness Centrality Growth

Definition:

$$\Delta_betweenness(T, \Delta t) = betweenness(T, t+\Delta t) - betweenness(T, t)$$

Measures: Does T become a critical bridge in the future graph?

Computation:

```
python

def betweenness_growth(theorem_T, dag, time_window=100):
    """
    How much does T's betweenness increase as library grows?
    """
    # Snapshot at theorem's creation
    G_initial = dag.subgraph_up_to_depth(theorem_T.depth)
    betweenness_initial = nx.betweenness centrality(G_initial)[theorem_T]

    # Snapshot after time_window new theorems
    G_later = dag.subgraph_up_to_depth(theorem_T.depth + time_window)
    betweenness_later = nx.betweenness centrality(G_later)[theorem_T]

    return betweenness_later - betweenness_initial
```

Why Important: High $\Delta_betweenness$ means T becomes essential infrastructure. Different from popularity.

Target 3: Tactic Innovation Score

Definition: Extent to which T introduces novel proof patterns

Computation:

```
python
```

```
def tactic_innovation(theorem_T, corpus_before):
    """
    Measure novelty of T's proof tactics compared to prior corpus.
    """
    # Build n-gram model of tactic sequences from all prior proofs
    tactic_ngram_model = build_ngram_model(
        [proof.tactic_sequence for proof in corpus_before],
        n=4 # 4-grams capture tactic combinations
    )

    # Compute perplexity of T's proof under this model
    T_tactics = theorem_T.proof.tactic_sequence
    log_prob = sum(
        log(tactic_ngram_model.probability(T_tactics[i:i+4]))
        for i in range(len(T_tactics) - 3)
    )
    perplexity = exp(-log_prob / len(T_tactics))

    # High perplexity = novel tactic combinations
    return perplexity
```

Refinement: Weight by downstream adoption

```
python

innovation_score = perplexity * num_theorems_adopting_pattern
```

Target 4: Module Connector Rank

Definition: Does T bridge previously disconnected research areas?

Computation:

```
python
```

```

def module_connector_rank(theorem_T, dag, module_assignments):
    """
    Measure how much T connects disparate modules.
    """
    # Get parents of T
    parents = dag.predecessors(theorem_T)
    parent_modules = [module_assignments[p] for p in parents]

    # Entropy of parent module distribution
    module_entropy = scipy.stats.entropy(
        np.bincount(parent_modules),
        base=2
    )

    # Get descendants of T
    descendants = dag.descendants(theorem_T)

    # Measure: does T's existence reduce modularity of future graph?
    G_without_T = dag.copy()
    G_without_T.remove_node(theorem_T)

    modularity_without = community_modularity(G_without_T)
    modularity_with = community_modularity(dag)

    # Negative  $\Delta$  modularity = T breaks down silos
    return {
        'parent_module_entropy': module_entropy,
        'modularity_reduction': modularity_without - modularity_with,
        'num_modules_connected': len(set(parent_modules))
    }

```

V. Feature Engineering for $r=0$ Prediction

Current Features (Keep):

- ✓ `depth`, `indeg_seen`
- ✓ `parent_*` statistics (degree, depth)
- ✓ `parentset_rarity`, `parent_diversity`
- ✓ `upstream_2hop_*` (non-local structure)

Add: Proof-Level Features

1. Proof Complexity Features

```
python

# Proof length statistics
'proof_length': len(tactic_sequence),
'proof_depth': max_nesting_depth(proof_tree),
'avg_premises_per_tactic': mean(len(t.premises) for t in tactics),

# Proof shape
'proof_branching_factor': mean(len(node.children) for node in proof_tree),
'proof_width': max(num_open_subgoals at any step),
```

2. Tactic Diversity Features

```
python

# Shannon entropy of tactic distribution in proof
'tactic_entropy': entropy([count(t) for t in unique_tactics]),

# Rare tactic usage
'rare_tactic_fraction': frac_tactics_below_1percentile,

# Tactic n-gram novelty (as in innovation score)
'tactic_4gram_perplexity': compute_perplexity(proof, ngram_model),
```

3. Semantic Abstraction Features

```
python

# Type signature generality
'uses_type_classes': count(type_class_constraints),
'type_variables': count(polymorphic_type_vars),
'abstraction_level': depth in type_hierarchy,

# Proof by: construction vs. contradiction vs. induction
'proof_mode': {constructive: 0, contradiction: 1, induction: 2},
```

4. Cross-Module Bridging Features

```
python
```

```

# Premise module diversity (already have parent_module_entropy)
# Add: premise THEOREM TYPE diversity
'premise_type_entropy': entropy([type_of(p) for p in all_premises]),

# Conceptual distance of parents
'parent_embedding_distance_mean': mean(pairwise_distances(
    [embed(p) for p in parents]
)),

# Where embed() comes from pre-trained theorem embeddings
# (e.g., Word2Vec on theorem statements, or GNN on full DAG)

```

5. Temporal/Position Features

```

python

# Is this theorem one of first in its module?
'module_age_rank': rank_within_module_by_creation_time,

# Frontier position
'betweenness_at_creation': betweenness centrality(G_seen)[T],
'closeness_to_recent': mean_distance_to_last_k_theorems,

```

Critical Addition: Pre-trained Embeddings

Learn graph representation on full DAG (allowed since it's unsupervised):

```

python

```

```

# Node2Vec or GraphSAGE on full theorem DAG
embeddings = train_graph_embedding(
    G_full_dag,
    dim=128,
    walk_length=80,
    context_size=10
)

# For theorem T at depth d, its embedding encodes:
# - Position in global graph structure
# - Similarity to other theorems (random walk co-occurrence)

# At prediction time (r=0), use embedding features:
'embedding_norm': np.linalg.norm(embeddings[T]),
'embedding_entropy': entropy(abs(embeddings[T])), # Spread across dimensions

# Distance to "hub" theorems (identified in full graph)
'distance_to_hubs': mean([
    cosine_distance(embeddings[T], embeddings[hub])
    for hub in top_100_hubs
])

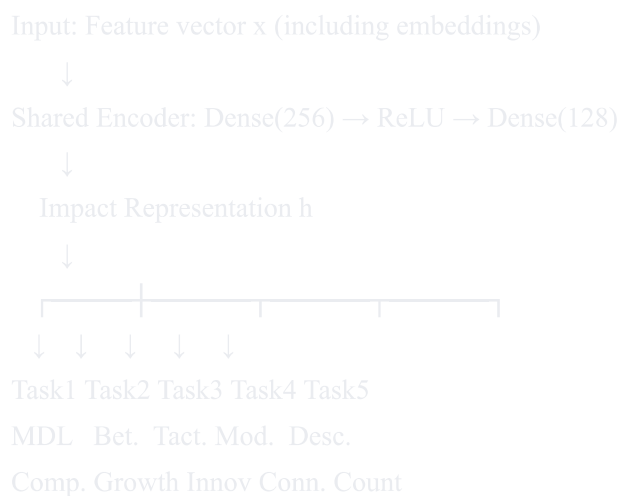
```

Key insight: Even at $r=0$, we can use embeddings from full graph *as features*. We're not leaking future links (those are targets), just global position.

VI. Multi-Task Learning Architecture

Don't predict one target. Predict multiple simultaneously—forces model to learn shared "impact" representation.

Proposed Architecture



↓ ↓ ↓ ↓ ↓
Output heads (regression or classification)

Loss Function:

```
python
```

```
L_total =  $\alpha_1$  · L_md1 +  $\alpha_2$  · L_betweenness +  $\alpha_3$  · L_tactic +  $\alpha_4$  · L_module +  $\alpha_5$  · L_descendants
```

Where weights α_i chosen by:

- Uncertainty-based weighting (higher weight to easier tasks initially)
- Or: task-specific validation loss balancing

Why This Works:

- MDL and betweenness both measure "structural importance"
- Tactic innovation and module connecting measure "conceptual novelty"
- Descendant count is noisy but high-signal for a few mega-hubs

Model learns: $\boxed{h}$ = latent "impact" that predicts all targets. More robust than single-task.

VII. Operationalizing "Interestingness"

Definition (Shannon + Zipf + Compression)

Interestingness Score:

```
I(T) = (novelty(T) · compression(T)) / complexity(T)
```

Component 1: Novelty

```
python
```

```
novelty(T) = KL_divergence(  
    P_tactics(T), # Tactic distribution in T's proof  
    P_tactics(corpus_before_T) # Background distribution  
)
```

High novelty = uses tactics in atypical combinations.

Component 2: Compression

```
python
```

```
compression(T) =  $\Delta\_MDL(T)$  # As defined in Section III
```

High compression = enables shorter proofs downstream.

Component 3: Complexity

```
python
```

```
complexity(T) = proof_length(T) + statement_length(T)
```

Normalize by theorem's own description length (elegance).

Empirical Validation

Hypothesis: High-I theorems are those human mathematicians cite as "elegant" or "fundamental".

Test:

1. Identify theorems with explicit recognition:
 - Named theorems (Pythagorean, Fundamental Theorem of Calculus)
 - Heavily commented in mathlib (often indicates significance)
 - Referenced in teaching materials
2. Compute $I(T)$ for all theorems
3. Check correlation: Do high-I theorems align with recognized important theorems?
4. Regression: Predict human "importance" ratings using $I(T)$ and components

Expected Result:

- Pure descendant count: weakly correlated with importance
- $I(T)$: strongly correlated (captures elegance + utility)

VIII. Adjacent Possible Dynamics

Growth Kernel Matrix

Definition: $K[i,j] = P(\text{theorem } j \text{ enables theorem } i \text{ in next } \Delta t \text{ steps})$

Estimation:

```
python
```

```

# For each pair (i, j) where i proved after j:
# Did j appear in i's proof?

K_empirical = np.zeros((n_theorems, n_theorems))

for i in range(n_theorems):
    proof_i = all_proofs[i]
    premises_i = extract_all_premises(proof_i)

    for j in premises_i:
        if j < i: # j proved before i
            time_gap = theorem_time[i] - theorem_time[j]

            # Kernel: decays with time gap
            K_empirical[i, j] = exp(-time_gap / tau)

# Normalize: each row sums to 1 (probability distribution)
K = K_empirical / K_empirical.sum(axis=1, keepdims=True)

```

Spectral Analysis:

```

python

eigenvalues, eigenvectors = np.linalg.eig(K)

# Leading eigenvector: "fertile" direction
# (which theorems, when proved, enable most future theorems)
fertility_score = eigenvectors[:, 0]

# Theorems with high fertility_score are high-impact

```

Prediction Task: Given features of theorem T at creation, predict its fertility_score.

Eigenvalue Acceleration Prediction

Insight: Some theorems shift the spectrum—accelerate research pace.

Measure:

```

python

```

```
def eigenvalue_acceleration(theorem_T, dag, window_before=50, window_after=50):
    """
    Does adding T change growth dynamics?
    """
    # Subgraph before T
    G_before = dag.nodes_in_time_window(T.time - window_before, T.time)
    K_before = compute_growth_kernel(G_before)
    λ_max_before = max(np.linalg.eigvals(K_before))

    # Subgraph after T
    G_after = dag.nodes_in_time_window(T.time, T.time + window_after)
    K_after = compute_growth_kernel(G_after)
    λ_max_after = max(np.linalg.eigvals(K_after))

    return λ_max_after - λ_max_before
```

Interpretation: Positive acceleration = T opens new research directions (increases fertility).

IX. Implementation Roadmap

Phase 1: Data Processing (Weeks 1-2)

- ☐ Parse all theorem proofs to extract tactic sequences
- ☐ Build tactic dependency trees for each proof
- ☐ Compute proof-level features (length, depth, tactic diversity)
- ☐ Create hash index for tactic subtree matching

Phase 2: MDL Computation (Weeks 3-4)

- ☐ Implement subgraph isomorphism matching (hash-based approximation)
- ☐ Compute Δ_MDL for all theorems (can parallelize)
- ☐ Validate: do high-MDL theorems align with intuitively important ones?
- ☐ Debug: check for degenerate cases (e.g., trivial lemmas with high descendant count but low compression)

Phase 3: Alternative Targets (Week 5)

- ☐ Compute betweenness growth for all theorems
- ☐ Compute tactic innovation scores
- ☐ Compute module connector ranks
- ☐ Correlation analysis: how do these targets relate?

Phase 4: Feature Engineering (Week 6)

- ☐ Add proof-level features to existing pipeline
- ☐ Train graph embeddings (Node2Vec) on full DAG
- ☐ Extract embedding-derived features
- ☐ Feature importance analysis (which features matter for which targets?)

Phase 5: Multi-Task Model (Weeks 7-8)

- ☐ Implement multi-task architecture (PyTorch/JAX)
- ☐ Train on joint prediction of {MDL, betweenness, innovation, descendants}
- ☐ Hyperparameter tuning (layer sizes, loss weights)
- ☐ Evaluate: does multi-task improve over single-task baselines?

Phase 6: Interestingness Metric (Week 9)

- ☐ Compute $I(T)$ for all theorems
- ☐ Identify ground-truth "important" theorems (named theorems, heavily cited)
- ☐ Validation: correlation with human importance
- ☐ Ablation: novelty vs. compression vs. complexity contributions

Phase 7: Adjacent Possible Analysis (Week 10)

- ☐ Build growth kernel matrix K
- ☐ Spectral analysis: fertility eigenvectors
- ☐ Predict fertility scores from $r=0$ features
- ☐ Eigenvalue acceleration analysis

Phase 8: Synthesis & Writing (Weeks 11-12)

- ☐ Compile results into paper/report
 - ☐ Visualizations: tactic innovation over time, MDL vs. descendants scatter, etc.
 - ☐ Connect to ATP literature (LEGO-Prover, tactic discovery)
 - ☐ Discussion: implications for AI-assisted theorem proving
-

X. Expected Outcomes & Impact

Immediate Results

1. **Better Prediction at $r=0$:** MDL-based targets should achieve $R^2 > 0.2$ (vs. current ~ 0)
2. **Interpretable Impact Metric:** $I(T)$ quantifies "interestingness" in operational terms
3. **Feature Importance:** Which proof characteristics predict structural impact?

Theoretical Contributions

1. **Formalization of Mathematical Value:** Beyond citations, what makes theorems important?
2. **Adjacent Possible Quantification:** Growth kernel provides mechanistic model of field evolution
3. **Compression ↔ Discovery Link:** Validates MDL principle in mathematical context

Practical Applications

1. **Proof Assistant Guidance:** Suggest "high-impact" lemmas to prove next
2. **Automated Theorem Generation:** Optimize for $I(T)$, not just correctness
3. **Curriculum Design:** Teach theorems in order of fertility (maximal enablement)

Connection to Broader Vision

This work provides **ground truth testbed** for:

- "Scientific intuition": Features that predict $I(T)$ operationalize intuition
- "Interesting problems": High $I(T)$ = interesting (novel + useful + elegant)
- "Discovery dynamics": Growth kernel models field evolution from first principles

Extends naturally to:

- Tactic discovery (from Xin et al.): MDL framework quantifies tactic value
- Proof search (ATP): Use $I(T)$ to prioritize subgoals
- Library growth (LEGO-Prover): Fertility eigenvectors identify high-value lemmas

XI. Technical Appendix: Code Scaffolding

A. Tactic Tree Construction

```
python
```

class **TacticTree**:

```
def __init__(self, proof_trace):  
    """  
    Build tree from Lean proof trace.  
  
    proof_trace: List of (tactic, premises, subgoals_before, subgoals_after)  
    """  
    self.root = TacticNode(tactic="root", premises=[], subgoals=[proof_trace[0].goal])  
    self._build_tree(self.root, proof_trace, 0)
```

```
def _build_tree(self, parent, trace, idx):  
    if idx >= len(trace):  
        return idx  
  
    tactic, premises, goals_before, goals_after = trace[idx]  
    node = TacticNode(tactic, premises, goals_after)  
    parent.add_child(node)  
  
    # For each subgoal, recursively build subtree  
    for goal in goals_after:  
        idx = self._build_tree(node, trace, idx + 1)  
  
    return idx
```

```
def subtrees(self, min_size=3):  
    """Enumerate all subtrees larger than min_size."""  
    def recurse(node):  
        if node.size() >= min_size:  
            yield node  
        for child in node.children:  
            yield from recurse(child)  
    return list(recurse(self.root))
```

class **TacticNode**:

```
def __init__(self, tactic, premises, subgoals):  
    self.tactic = tactic  
    self.premises = premises  
    self.subgoals = subgoals  
    self.children = []  
  
def add_child(self, node):  
    self.children.append(node)
```

```
def size(self):
    return 1 + sum(c.size() for c in self.children)

def signature(self, depth=3):
    """Canonical signature for isomorphism."""
    if depth == 0:
        return hash(self.tactic)

    return hash((
        self.tactic,
        frozenset(p.theorem_type for p in self.premises),
        tuple(sorted(c.signature(depth-1) for c in self.children))
    ))
```

B. MDL Core Function

python

```

def compute_mdل_compression_fast(theorem_id, dag, proofs, hash_index):
    """
    Fast MDL computation using hash index.

    Returns dict with compression statistics.
    """
    # Get theorem's proof tactic tree
    theorem_tree = TacticTree(proofs[theorem_id])

    # Extract and hash all subtrees
    theorem_subtrees = theorem_tree.subtrees(min_size=3)
    theorem_hashes = [st.signature() for st in theorem_subtrees]

    # Find compression sites
    compression_sites = []
    for h in theorem_hashes:
        if h in hash_index:
            # This subtree pattern appears in other proofs
            for (other_theorem_id, location) in hash_index[h]:
                if dag.is_descendant(other_theorem_id, theorem_id):
                    # other_theorem proved AFTER this theorem
                    compression_sites.append({
                        'theorem': other_theorem_id,
                        'location': location,
                        'pattern_size': theorem_subtrees[theorem_hashes.index(h)].size()
                    })

    # Compute compression value
    gross_compression = 0
    for site in compression_sites:
        # Original cost: all tactics in pattern
        original_cost = site['pattern_size'] * AVG_TACTIC_COST

        # Replacement cost: single "apply theorem_id"
        new_cost = APPLY_COST + log2(len(hash_index)) # Premise selection

        gross_compression += (original_cost - new_cost)

    # Subtract theorem's own cost
    theorem_cost = theorem_tree.root.size() * AVG_TACTIC_COST
    net_compression = gross_compression - theorem_cost

    return {

```

```
'theorem_id': theorem_id,
'gross_compression': gross_compression,
'theorem_cost': theorem_cost,
'net_compression': net_compression,
'num_sites': len(compression_sites),
'descendants_affected': len(set(s['theorem'] for s in compression_sites))
}
```

C. Feature Extraction Pipeline

python

```

class ProofFeatureExtractor:
    def __init__(self, all_proofs, dag, ngram_model):
        self.all_proofs = all_proofs
        self.dag = dag
        self.ngram_model = ngram_model

    def extract(self, theorem_id):
        """Extract all features for a theorem."""
        proof = self.all_proofs[theorem_id]
        tree = TacticTree(proof)

        features = {}

        # Basic proof stats
        features['proof_length'] = tree.root.size()
        features['proof_depth'] = tree.depth()
        features['proof_width'] = tree.max_width()

        # Tactic diversity
        tactics = tree.all_tactics()
        features['tactic_entropy'] = scipy.stats.entropy(
            np.bincount([t.id for t in tactics])
        )
        features['unique_tactics'] = len(set(tactics))

        # Tactic innovation (perplexity)
        features['tactic_4gram_perplexity'] = self.ngram_model.perplexity(tactics)

        # Premise features
        all_premises = tree.all_premises()
        features['total_premises'] = len(all_premises)
        features['unique_premises'] = len(set(all_premises))
        features['avg_premises_per_tactic'] = len(all_premises) / features['proof_length']

        # Parent module diversity (if modules available)
        parents = self.dag.predecessors(theorem_id)
        parent_modules = [self.dag.nodes[p]['module'] for p in parents]
        features['parent_module_entropy'] = scipy.stats.entropy(
            np.bincount(parent_modules)
        )

        # Add more features as needed...

```

XII. Open Questions & Future Directions

1. **Compression Approximation Quality:** How accurate is hash-based matching vs. true proof synthesis?
 - Test on subset: compare our approximation to LLM-generated alternative proofs
 2. **Temporal Stability:** Do fertility eigenvectors change as field matures?
 - Track spectral evolution over time windows
 3. **Transfer to Other Domains:** Does MDL framework work for code libraries? Physical theories?
 - Test on: theorem proving (done), software repositories (next), scientific papers (future)
 4. **Causality:** Correlation $\neq$ causation. Do high-I(T) theorems *cause* field acceleration?
 - Natural experiments: compare parallel developments in different proof assistant communities
 5. **Optimizing for Discovery:** If we build proof search guided by predicted I(T), do we discover different (better?) theorems?
 - RL with I(T) as reward signal
-

End of Plan Document v1.0

Next steps: Review, refine, begin Phase 1 implementation.